

Guide Professionnel de la Conteneurisation

[DOCKER]

DOCKER

Guide Complet de la Conteneurisation

Installation • Images • Conteneurs • Réseaux
Volumes • Docker Compose • Sécurité • Production

EL KHLIFI Imad

PhDc / Data Scientist / Data Analyst / AI

5 mars 2026

*“La conteneurisation n’est pas seulement une technologie,
c’est une philosophie du développement moderne.”*

*Dédié à tous les développeurs et ingénieurs DevOps
qui cherchent à maîtriser l’art de la conteneurisation.*

Table des matières

Avant-Propos	6
1 Introduction à la Conteneurisation	7
1.1 Qu'est-ce que la virtualisation ?	7
1.1.1 Les machines virtuelles (VM)	7
1.2 Les conteneurs : une révolution	8
1.2.1 Conteneur vs Machine Virtuelle	8
1.3 Historique et évolution de Docker	8
1.3.1 Chronologie Docker	8
1.4 Les technologies sous-jacentes	8
1.4.1 Les namespaces Linux	9
1.4.2 Les cgroups (Control Groups)	9
1.4.3 Union File System (UnionFS)	9
2 Installation et Configuration de Docker	10
2.1 Architecture de Docker	10
2.1.1 Docker Engine	10
2.2 Installation sur Linux (Ubuntu/Debian)	10
2.2.1 Méthode recommandée : dépôt officiel Docker	10
2.2.2 Configuration post-installation	11
2.3 Installation sur Windows et macOS	12
2.3.1 Docker Desktop	12
2.3.2 Configuration système requise	12
2.4 Vérification de l'installation	12
3 Images Docker	13
3.1 Comprendre les images Docker	13
3.2 Docker Hub : le registre d'images	13
3.2.1 Catégories d'images	13
3.2.2 Commandes de gestion des images	13
3.3 Nomenclature des images	14
3.4 Créer une image personnalisée : le Dockerfile	14
3.4.1 Instructions fondamentales du Dockerfile	14
3.4.2 Exemple complet : Application Node.js	15
3.4.3 Construire et tagger une image	16
3.5 Optimisation des images Docker	17
3.5.1 Multi-stage builds	17

3.5.2	Bonnes pratiques pour les Dockerfiles	17
4	Gestion des Conteneurs	19
4.1	Cycle de vie d'un conteneur	19
4.2	Commandes essentielles	19
4.2.1	Créer et démarrer des conteneurs	19
4.2.2	Inspecter et surveiller les conteneurs	20
4.2.3	Interagir avec les conteneurs	20
4.2.4	Arrêter et supprimer des conteneurs	21
4.3	Options avancées de <code>docker run</code>	22
5	Réseaux Docker	23
5.1	Introduction aux réseaux Docker	23
5.2	Types de réseaux Docker	23
5.2.1	Bridge (pont) réseau par défaut	23
5.2.2	Host réseau hôte	24
5.2.3	None pas de réseau	24
5.3	Gestion des réseaux	24
6	Volumes et Persistance des Données	26
6.1	Le problème de la persistance	26
6.2	Types de stockage Docker	26
6.2.1	Volumes Docker (recommandé)	26
6.2.2	Bind Mounts (montages liés)	27
6.2.3	tmpfs (mémoire temporaire)	27
6.3	Stratégies de sauvegarde	27
7	Docker Compose	29
7.1	Introduction à Docker Compose	29
7.2	Structure d'un fichier docker-compose.yml	29
7.3	Exemple complet : Stack Web LAMP	30
7.4	Commandes Docker Compose	31
8	Registres Docker et Distribution d'Images	33
8.1	Docker Hub	33
8.1.1	Publier une image sur Docker Hub	33
8.2	Autres registres	33
8.2.1	GitHub Container Registry (GHCR)	33
8.2.2	Registre privé auto-hébergé	34
9	Sécurité Docker	35
9.1	Principes de sécurité fondamentaux	35
9.2	Sécurité des images	35
9.2.1	Utiliser des images de confiance	35
9.3	Sécurité des conteneurs	36
9.3.1	Principe du moindre privilège	36
9.3.2	Limiter les ressources	36
9.4	Sécurité du daemon Docker	36

10 Docker en Production	38
10.1 Bonnes pratiques de production	38
10.2 Logging et monitoring	38
10.2.1 Configuration des logs	38
10.2.2 Stack de monitoring : Prometheus + Grafana	38
10.3 Haute disponibilité avec Docker Swarm	39
10.4 CI/CD avec Docker	40
10.4.1 Intégration dans GitHub Actions	40
11 Cas Pratiques et Exercices	42
11.1 TP1 : Déployer un site web statique	42
11.2 TP2 : Stack complète avec Docker Compose	43
11.3 TP3 : Exportation et portabilité	45
12 Référence des Commandes et Glossaire	46
12.1 Cheat Sheet Docker	47
12.2 Glossaire Docker	47
12.3 Ressources et liens utiles	48

Avant-Propos

[i] Information

Ce guide a été rédigé par **EL KHLIFI Imad** avec pour objectif de fournir une ressource complète et pratique sur Docker, allant des concepts fondamentaux jusqu'aux déploiements avancés en production.

Docker a révolutionné la manière dont nous développons, livrons et exécutons les applications. Depuis son lancement en 2013 par Solomon Hykes chez dotCloud, Docker est devenu le standard de facto pour la conteneurisation d'applications, adoptée par des millions de développeurs et d'entreprises à travers le monde.

Ce livre s'adresse aussi bien aux débutants souhaitant découvrir la conteneurisation qu'aux praticiens expérimentés cherchant à approfondir leurs connaissances. Chaque chapitre est structuré de manière progressive, avec de nombreux exemples pratiques, des commandes commentées et des cas d'usage réels.

Ce que vous apprendrez dans ce livre :

- Les fondamentaux de la virtualisation et de la conteneurisation
- L'installation et la configuration de Docker sur différentes plateformes
- La création et la gestion d'images Docker personnalisées
- L'orchestration de conteneurs avec Docker Compose
- La gestion des réseaux et des volumes Docker
- Les bonnes pratiques de sécurité
- Le déploiement en environnements de production

Prérequis :

Une connaissance de base des systèmes Linux et des commandes en ligne de commande est recommandée. Aucune expérience préalable avec Docker n'est nécessaire.

EL KHLIFI Imad
PhDc

Chapitre 1

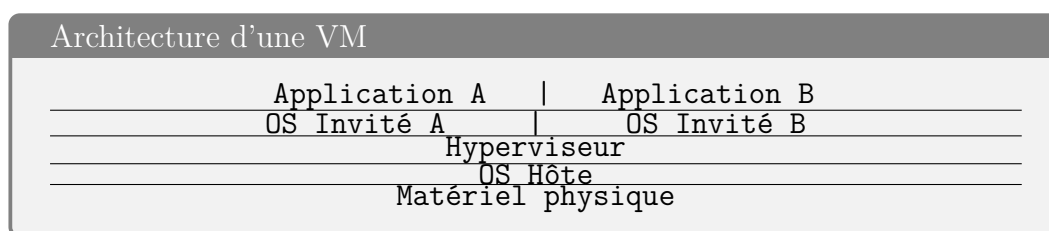
Introduction à la Conteneurisation

1.1 Qu'est-ce que la virtualisation ?

Avant d'aborder Docker, il est essentiel de comprendre la notion de **virtualisation**. La virtualisation est une technologie qui permet de créer des versions virtuelles de ressources informatiques, notamment des serveurs, des systèmes de stockage, des réseaux et même des systèmes d'exploitation.

1.1.1 Les machines virtuelles (VM)

Une **machine virtuelle** est une émulation complète d'un ordinateur physique. Elle inclut un système d'exploitation complet, des pilotes virtuels, et toutes les applications nécessaires. Les hyperviseurs (comme VMware, VirtualBox ou KVM) permettent de faire tourner plusieurs VM sur une même machine physique.



Avantages des VM :

- Isolation complète entre les environnements
- Possibilité de faire tourner différents systèmes d'exploitation
- Snapshots et sauvegardes faciles

Inconvénients des VM :

- Consommation importante de ressources (RAM, CPU, stockage)
- Temps de démarrage long (plusieurs minutes)
- Duplication du système d'exploitation
- Coût de licence pour certains OS

1.2 Les conteneurs : une révolution

Les **conteneurs** représentent une approche différente de l'isolation. Au lieu d'émuler un ordinateur complet, les conteneurs partagent le noyau du système d'exploitation hôte et isolent uniquement les processus, les fichiers système et les ressources réseau.

[i] Information

Un conteneur est une unité légère et portable qui encapsule une application et toutes ses dépendances (bibliothèques, fichiers de configuration, variables d'environnement) dans un environnement isolé.

1.2.1 Conteneur vs Machine Virtuelle

Critère	Machine Virtuelle	Conteneur
Taille	Plusieurs Go (OS complet)	Quelques Mo à quelques centaines de Mo
Démarrage	Minutes	Secondes
Isolation	Complète (hyperviseur)	Partage du noyau OS
Performance	Overhead important	Quasi-natif
Portabilité	Limitée	Excellente
OS requis	OS invité complet	Partage l'OS hôte

1.3 Historique et évolution de Docker

Docker a été créé par **Solomon Hykes** et présenté pour la première fois en mars 2013 lors de la conférence PyCon. Initialement développé comme un projet interne de la société dotCloud, Docker a rapidement suscité un engouement mondial.

1.3.1 Chronologie Docker

- **2013** : Lancement de Docker en open source
- **2014** : Docker 1.0 première version stable. La société devient Docker Inc.
- **2015** : Introduction de Docker Compose et Docker Swarm
- **2016** : Docker for Mac et Docker for Windows
- **2017** : Kubernetes intégré dans Docker Enterprise
- **2019** : Docker Desktop amélioré, focus sur les développeurs
- **2020+** : Adoption massive dans les environnements cloud et CI/CD

1.4 Les technologies sous-jacentes

Docker repose sur plusieurs technologies Linux fondamentales :

1.4.1 Les namespaces Linux

Les **namespaces** permettent d'isoler les ressources système. Docker utilise plusieurs types de namespaces :

- **pid** : Isolation des processus
- **net** : Isolation réseau
- **mnt** : Isolation du système de fichiers
- **uts** : Isolation du hostname
- **ipc** : Isolation des communications inter-processus
- **user** : Isolation des utilisateurs et groupes

1.4.2 Les cgroups (Control Groups)

Les **cgroups** permettent de limiter et contrôler les ressources allouées aux processus : CPU, mémoire, I/O disque, réseau. Grâce aux cgroups, Docker peut garantir qu'un conteneur ne consomme pas toutes les ressources du système hôte.

1.4.3 Union File System (UnionFS)

Docker utilise un système de fichiers en couches (**layers**) grâce à UnionFS (overlay2, AUFS, etc.). Chaque instruction dans un Dockerfile crée une nouvelle couche immuable. Ces couches sont partagées entre les images, réduisant ainsi l'espace disque utilisé.

Chapitre 2

Installation et Configuration de Docker

2.1 Architecture de Docker

Avant d'installer Docker, comprenons son architecture :

2.1.1 Docker Engine

Le **Docker Engine** est le cur du système. Il se compose de :

- **Docker Daemon (dockerd)** : Le processus serveur qui gère les conteneurs, images, réseaux et volumes
- **REST API** : Interface de communication entre le client et le daemon
- **Docker CLI** : L'interface en ligne de commande utilisée par les développeurs

[i] Architecture Client-Serveur

Docker utilise une architecture client-serveur. Le client Docker communique avec le daemon Docker via une API REST, soit localement (socket Unix), soit à distance (TCP).

2.2 Installation sur Linux (Ubuntu/Debian)

2.2.1 Méthode recommandée : dépôt officiel Docker

```
1 # 1. Mettre a jour les paquets
2 sudo apt update
3
4 # 2. Installer les dependances
5 sudo apt install -y \
6     ca-certificates \
7     curl \
8     gnupg \
```

```
9      lsb-release
10
11 # 3. Ajouter la cle GPG officielle de Docker
12 sudo mkdir -p /etc/apt/keyrings
13 curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
14     | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
15
16 # 4. Configurer le depot stable
17 echo \
18     "deb [arch=$(dpkg --print-architecture) \
19     signed-by=/etc/apt/keyrings/docker.gpg] \
20     https://download.docker.com/linux/ubuntu \
21     $(lsb_release -cs) stable" \
22     | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
23
24 # 5. Installer Docker Engine
25 sudo apt update
26 sudo apt install -y docker-ce docker-ce-cli \
27     containerd.io docker-compose-plugin
28
29 # 6. Verifier l'installation
30 docker --version
31 docker compose version
```

Listing 2.1 – Installation de Docker sur Ubuntu

2.2.2 Configuration post-installation

```
1 # Demarrer et activer Docker au demarrage
2 sudo systemctl start docker
3 sudo systemctl enable docker
4
5 # Ajouter l'utilisateur au groupe docker
6 # (evite d'utiliser sudo pour chaque commande)
7 sudo usermod -aG docker $USER
8
9 # Reconnexion necessaire pour appliquer les changements
10 newgrp docker
11
12 # Tester sans sudo
13 docker run hello-world
```

Listing 2.2 – Configuration après installation

[!] Avertissement

Ajouter un utilisateur au groupe `docker` lui donne des privilèges équivalents à `root`. Faites attention dans les environnements de production.

2.3 Installation sur Windows et macOS

2.3.1 Docker Desktop

Pour Windows et macOS, Docker propose **Docker Desktop**, une application graphique qui inclut :

- Docker Engine
- Docker CLI
- Docker Compose
- Kubernetes (optionnel)
- Une interface graphique de gestion

2.3.2 Configuration système requise

Système	OS minimum	RAM minimum
Windows	Windows 10 64-bit (Build 19041+)	4 Go
macOS	macOS 11 (Big Sur)	4 Go
Linux	Kernel 3.10+	2 Go

2.4 Vérification de l'installation

```
1 # Version de Docker
2 docker version
3
4 # Informations systeme Docker
5 docker info
6
7 # Tester avec l'image hello-world
8 docker run hello-world
9
10 # Lister les images locales
11 docker images
12
13 # Lister les conteneurs en cours
14 docker ps
```

Listing 2.3 – Vérification complète de l'installation

Chapitre 3

Images Docker

3.1 Comprendre les images Docker

Une **image Docker** est un modèle en lecture seule utilisé pour créer des conteneurs. Elle contient le système de fichiers de l'application, ses dépendances, les variables d'environnement et les configurations nécessaires.

[i] Information

Les images Docker sont construites en **couches (layers)** superposées. Chaque couche représente un ensemble de changements dans le système de fichiers. Cette approche permet de réutiliser les couches communes entre plusieurs images, économisant ainsi de l'espace disque.

3.2 Docker Hub : le registre d'images

Docker Hub est le registre public officiel d'images Docker. Il contient des millions d'images officielles et communautaires.

3.2.1 Catégories d'images

- **Images officielles** : Maintenues par Docker Inc. (ubuntu, nginx, mysql, python, etc.)
- **Images vérifiées** : Publiées par des éditeurs certifiés
- **Images communautaires** : Publiées par la communauté (format : utilisateur/image)

3.2.2 Commandes de gestion des images

```
1 # Rechercher une image sur Docker Hub
2 docker search nginx
3 docker search --filter is-official=true python
4
5 # Telecharger une image
6 docker pull ubuntu
```

```
7 docker pull ubuntu:22.04           # version spécifique
8 docker pull nginx:alpine           # variante legere
9
10 # Lister les images locales
11 docker images
12 docker image ls
13
14 # Inspecter une image
15 docker image inspect nginx
16
17 # Voir l'historique des couches
18 docker history nginx
19
20 # Supprimer une image
21 docker rmi nginx
22 docker image rm ubuntu:22.04
23
24 # Supprimer toutes les images non utilisees
25 docker image prune
26 docker image prune -a              # toutes les images sans
                                     conteneur
```

Listing 3.1 – Gestion des images Docker

3.3 Nomenclature des images

Le format complet d'une image Docker est :

```
[registre/] [namespace/] nom-image: [tag]
```

Exemples :

- nginx image officielle, tag latest
- nginx:1.25-alpine version spécifique, variante Alpine
- moncompte/monapp:v2.0 image personnelle avec tag
- ghcr.io/org/app:main registre GitHub

3.4 Créer une image personnalisée : le Dockerfile

Un **Dockerfile** est un fichier texte contenant une série d'instructions pour construire une image Docker.

3.4.1 Instructions fondamentales du Dockerfile

```
1 # FROM : image de base
2 FROM ubuntu:22.04
3
4 # LABEL : metadonnees de l'image
5 LABEL maintainer="EL KHLIFI Imad <elkhlifi@example.com>"
```

```

6 LABEL version="1.0"
7 LABEL description="Application web de demonstration"
8
9 # ARG : argument de construction (non persistant)
10 ARG APP_VERSION=1.0
11
12 # ENV : variable d'environnement (persistante)
13 ENV NODE_ENV=production
14 ENV APP_PORT=3000
15
16 # RUN : executer une commande lors de la construction
17 RUN apt-get update && apt-get install -y \
18     curl \
19     wget \
20     vim \
21     && rm -rf /var/lib/apt/lists/*
22
23 # WORKDIR : definir le repertoire de travail
24 WORKDIR /app
25
26 # COPY : copier des fichiers depuis l'hote
27 COPY package.json ./
28 COPY . .
29
30 # ADD : similaire a COPY, supporte URLs et archives
31 ADD https://example.com/file.tar.gz /tmp/
32
33 # EXPOSE : documenter le port expose
34 EXPOSE 3000
35
36 # VOLUME : definir un point de montage
37 VOLUME ["/data", "/logs"]
38
39 # USER : changer d'utilisateur
40 RUN useradd -m appuser
41 USER appuser
42
43 # CMD : commande par defaut (remplacable)
44 CMD ["node", "server.js"]
45
46 # ENTRYPOINT : point d'entree fixe
47 ENTRYPOINT ["node"]
48 CMD ["server.js"]

```

Listing 3.2 – Instructions Dockerfile essentielles

3.4.2 Exemple complet : Application Node.js

```

1 # Utiliser l'image officielle Node.js LTS
2 FROM node:18-alpine
3

```

```
4 # Définir le mainteneur
5 LABEL maintainer="EL KHLIFI Imad"
6
7 # Créer et définir le repertoire de travail
8 WORKDIR /usr/src/app
9
10 # Copier les fichiers de dependances en premier
11 # (optimisation du cache Docker)
12 COPY package*.json ./
13
14 # Installer les dependances
15 RUN npm ci --only=production
16
17 # Copier le reste du code source
18 COPY . .
19
20 # Créer un utilisateur non-root pour la securite
21 RUN addgroup -S appgroup && \
22     adduser -S appuser -G appgroup
23 USER appuser
24
25 # Exposer le port de l'application
26 EXPOSE 8080
27
28 # Healthcheck
29 HEALTHCHECK --interval=30s --timeout=3s \
30     CMD wget --no-verbose --tries=1 \
31         --spider http://localhost:8080/ || exit 1
32
33 # Demarrer l'application
34 CMD ["node", "server.js"]
```

Listing 3.3 – Dockerfile pour une application Node.js

3.4.3 Construire et tagger une image

```
1 # Construction basique
2 docker build -t mon-app .
3
4 # Construction avec tag complet
5 docker build -t elkhilifi/mon-app:v1.0 .
6
7 # Construction avec argument
8 docker build \
9     --build-arg APP_VERSION=2.0 \
10     -t mon-app:v2.0 \
11     -f Dockerfile.prod \
12     .
13
14 # Lister les images creees
15 docker images | grep mon-app
```


Listing 3.4 – Construction d’une image

3.5 Optimisation des images Docker

3.5.1 Multi-stage builds

Les **builds multi-étapes** permettent de créer des images de production légères en séparant l’environnement de construction de l’environnement d’exécution.

```
1 # === STAGE 1 : Construction ===
2 FROM golang:1.21 AS builder
3
4 WORKDIR /src
5 COPY go.* ./
6 RUN go mod download
7
8 COPY . .
9 RUN CGO_ENABLED=0 GOOS=linux \
10     go build -o /app ./cmd/server
11
12 # === STAGE 2 : Production ===
13 FROM alpine:3.18 AS production
14
15 # Securite : certificats SSL
16 RUN apk --no-cache add ca-certificates
17
18 WORKDIR /root/
19
20 # Copier UNIQUEMENT le binaire compile
21 COPY --from=builder /app ./app
22
23 EXPOSE 8080
24
25 CMD ["/app"]
```

Listing 3.5 – Multi-stage build pour une application Go

[*] Astuce

Le multi-stage build peut réduire la taille d’une image de plusieurs centaines de Mo à quelques Mo. Par exemple, une image Go passe de 900 Mo (avec le SDK) à 10 Mo (binaire seul dans Alpine).

3.5.2 Bonnes pratiques pour les Dockerfiles

1. **Minimiser le nombre de couches** : Combiner les commandes RUN avec &&
2. **Utiliser .dockerignore** : Exclure les fichiers inutiles
3. **Choisir des images de base légères** : Alpine, Distroless, Slim

4. **Mettre en cache efficacement** : Copier package.json avant le code source
5. **Ne pas exécuter en root** : Utiliser un utilisateur non-privilégié
6. **Utiliser des tags spécifiques** : Éviter :latest en production

```
1 # Git
2 .git
3 .gitignore
4
5 # Documentation
6 README.md
7 docs/
8
9 # Logs et temporaires
10 *.log
11 tmp/
12 .tmp/
13
14 # Dependances (installée dans le conteneur)
15 node_modules/
16 vendor/
17
18 # Tests
19 *.test.js
20 tests/
21 coverage/
22
23 # CI/CD
24 .github/
25 .gitlab-ci.yml
26 Jenkinsfile
27
28 # Environnements locaux
29 .env.local
30 .env.*.local
```

Listing 3.6 – Exemple de fichier .dockerignore

Chapitre 4

Gestion des Conteneurs

4.1 Cycle de vie d'un conteneur

Un conteneur Docker passe par plusieurs états au cours de son cycle de vie :

created → running → paused → stopped → removed

4.2 Commandes essentielles

4.2.1 Créer et démarrer des conteneurs

```
1 # Lancer un conteneur de maniere interactive
2 docker run -it ubuntu bash
3
4 # Lancer en arriere-plan (detache)
5 docker run -d nginx
6
7 # Avec un nom personnalise
8 docker run -d --name mon-serveur nginx
9
10 # Avec mapping de port (hote:conteneur)
11 docker run -d -p 8080:80 --name web nginx
12
13 # Avec variable d'environnement
14 docker run -d \
15     -e MYSQL_ROOT_PASSWORD=secret \
16     -e MYSQL_DATABASE=mydb \
17     --name mysql-server \
18     mysql:8.0
19
20 # Supprimer automatiquement apres arret
21 docker run --rm ubuntu echo "Hello Docker"
22
23 # Limiter les ressources
24 docker run -d \
25     --memory="512m" \
```

```
26     --cpus="0.5" \  
27     --name limites \  
28     nginx  
29  
30 # Lancer avec politique de redemarrage  
31 docker run -d \  
32     --restart=always \  
33     --name serveur-prod \  
34     nginx
```

Listing 4.1 – Commandes de création et démarrage

4.2.2 Inspecter et surveiller les conteneurs

```
1 # Lister les conteneurs actifs  
2 docker ps  
3  
4 # Lister tous les conteneurs  
5 docker ps -a  
6  
7 # Statistiques en temps reel  
8 docker stats  
9  
10 # Statistiques d'un conteneur specifique  
11 docker stats mon-serveur  
12  
13 # Logs d'un conteneur  
14 docker logs mon-serveur  
15  
16 # Logs en temps reel  
17 docker logs -f mon-serveur  
18  
19 # Dernières 50 lignes de log avec timestamp  
20 docker logs --tail=50 --timestamps mon-serveur  
21  
22 # Inspecter la configuration complete  
23 docker inspect mon-serveur  
24  
25 # Processus dans le conteneur  
26 docker top mon-serveur
```

Listing 4.2 – Monitoring des conteneurs

4.2.3 Interagir avec les conteneurs

```
1 # Executer une commande dans un conteneur actif  
2 docker exec mon-serveur ls /usr/share/nginx/html  
3  
4 # Ouvrir un terminal interactif  
5 docker exec -it mon-serveur bash  
6
```

```
7 # Copier un fichier vers le conteneur
8 docker cp monindex.html mon-serveur:/usr/share/nginx/html/
9
10 # Copier un fichier depuis le conteneur
11 docker cp mon-serveur:/etc/nginx/nginx.conf ./nginx.conf
12
13 # Voir les differences par rapport a l'image
14 docker diff mon-serveur
```

Listing 4.3 – Interaction avec les conteneurs

4.2.4 Arrêter et supprimer des conteneurs

```
1 # Arrêter proprement (SIGTERM)
2 docker stop mon-serveur
3
4 # Arrêter immédiatement (SIGKILL)
5 docker kill mon-serveur
6
7 # Redemarrer
8 docker restart mon-serveur
9
10 # Mettre en pause
11 docker pause mon-serveur
12 docker unpause mon-serveur
13
14 # Supprimer un conteneur arrêté
15 docker rm mon-serveur
16
17 # Forcer la suppression (même si actif)
18 docker rm -f mon-serveur
19
20 # Supprimer tous les conteneurs arrêtés
21 docker container prune
22
23 # Tout nettoyer
24 docker system prune
25 docker system prune -a # images aussi
```

Listing 4.4 – Arrêt et suppression

4.3 Options avancées de docker run

Option	Description
-d	Mode détaché (arrière-plan)
-it	Mode interactif avec pseudo-TTY
-name	Donner un nom au conteneur
-p	Mapping de port <code>hôte:conteneur</code>
-P	Mapper tous les ports exposés aléatoirement
-v	Monter un volume
-e	Variable d'environnement
-env-file	Charger les variables depuis un fichier
-network	Connecter à un réseau
-rm	Supprimer après arrêt
-restart	Politique de redémarrage
-memory	Limiter la mémoire
-cpus	Limiter les CPU
-user	Spécifier l'utilisateur
-workdir	Répertoire de travail
-hostname	Nom d'hôte du conteneur

Chapitre 5

Réseaux Docker

5.1 Introduction aux réseaux Docker

Docker offre un système de réseau complet permettant aux conteneurs de communiquer entre eux et avec l'extérieur. Par défaut, Docker crée trois réseaux lors de son installation.

```
1 docker network ls
2
3 # Sortie typique :
4 # NETWORK ID          NAME           DRIVER        SCOPE
5 # abc123def456        bridge        bridge        local
6 # ghi789jkl012        host          host          local
7 # mno345pqr678        none          null          local
```

Listing 5.1 – Lister les réseaux Docker

5.2 Types de réseaux Docker

5.2.1 Bridge (pont) réseau par défaut

Le réseau **bridge** est le mode par défaut. Les conteneurs sur le même réseau bridge peuvent communiquer entre eux. Docker crée une interface réseau virtuelle (**docker0**) sur l'hôte.

```
1 # Créer un réseau bridge personnalisée
2 docker network create mon-reseau
3
4 # Créer avec des options avancées
5 docker network create \
6     --driver bridge \
7     --subnet 172.20.0.0/16 \
8     --ip-range 172.20.240.0/20 \
9     --gateway 172.20.0.1 \
10    mon-reseau-custom
11
```

```
12 # Lancer des conteneurs sur ce reseau
13 docker run -d --name app1 --network mon-reseau nginx
14 docker run -d --name app2 --network mon-reseau nginx
15
16 # Les conteneurs peuvent se pinguer par nom !
17 docker exec app1 ping app2
```

Listing 5.2 – Utilisation du réseau bridge

[*] Astuce

Contrairement au réseau bridge par défaut, les réseaux bridge personnalisés offrent la **résolution DNS automatique** par nom de conteneur. C'est pourquoi il est recommandé de toujours créer des réseaux dédiés.

5.2.2 Host réseau hôte

En mode **host**, le conteneur partage directement la pile réseau de l'hôte. Pas d'isolation réseau, mais performances maximales.

```
1 # Le conteneur utilise le reseau de l'hote directement
2 docker run -d --network host nginx
3
4 # Le port 80 du conteneur est directement le port 80 de l'
   hote
5 # Pas besoin de -p 80:80
```

Listing 5.3 – Mode réseau host

5.2.3 None pas de réseau

Le mode **none** désactive complètement le réseau du conteneur. Utile pour les traitements isolés.

```
1 docker run --network none ubuntu ip addr
```

Listing 5.4 – Mode réseau none

5.3 Gestion des réseaux

```
1 # Lister les reseaux
2 docker network ls
3
4 # Inspecter un reseau
5 docker network inspect mon-reseau
6
7 # Connecter un conteneur a un reseau
8 docker network connect mon-reseau mon-conteneur
9
10 # Deconnecter
```



```
11 docker network disconnect mon-reseau mon-conteneur
12
13 # Supprimer un reseau
14 docker network rm mon-reseau
15
16 # Nettoyer les reseaux non utilises
17 docker network prune
```

Listing 5.5 – Commandes de gestion réseau

Chapitre 6

Volumes et Persistance des Données

6.1 Le problème de la persistance

Par défaut, les données écrites dans un conteneur sont **éphémères** : elles disparaissent lorsque le conteneur est supprimé. Pour persister des données, Docker propose trois mécanismes.

6.2 Types de stockage Docker

6.2.1 Volumes Docker (recommandé)

Les **volumes Docker** sont gérés entièrement par Docker et stockés dans `/var/lib/docker/volumes/` sur l'hôte. Ils sont la solution recommandée pour la persistance.

```
1 # Créer un volume
2 docker volume create mes-donnees
3
4 # Lister les volumes
5 docker volume ls
6
7 # Inspecter un volume
8 docker volume inspect mes-donnees
9
10 # Utiliser un volume dans un conteneur
11 docker run -d \
12     --name mysql-db \
13     -v mes-donnees:/var/lib/mysql \
14     -e MYSQL_ROOT_PASSWORD=secret \
15     mysql:8.0
16
17 # Partager un volume entre conteneurs
18 docker run -d --name app1 -v shared-data:/data nginx
19 docker run -d --name app2 -v shared-data:/data nginx
20
21 # Supprimer un volume
22 docker volume rm mes-donnees
```

```
23
24 # Supprimer les volumes non utilises
25 docker volume prune
```

Listing 6.1 – Gestion des volumes Docker

6.2.2 Bind Mounts (montages liés)

Les **bind mounts** lient directement un répertoire de l'hôte à un répertoire du conteneur. Pratiques pour le développement.

```
1 # Syntaxe avec -v
2 docker run -d \
3     --name dev-web \
4     -v /home/elkhlifi/projet:/var/www/html \
5     -p 8080:80 \
6     nginx
7
8 # Syntaxe moderne avec --mount
9 docker run -d \
10    --name dev-web \
11    --mount type=bind,\
12           source=/home/elkhlifi/projet,\
13           target=/var/www/html \
14    -p 8080:80 \
15    nginx
16
17 # Lecture seule
18 docker run -d \
19     -v /etc/nginx.conf:/etc/nginx/nginx.conf:ro \
20     nginx
```

Listing 6.2 – Utilisation des bind mounts

6.2.3 tmpfs (mémoire temporaire)

Les montages **tmpfs** stockent les données en mémoire RAM. Données perdues à l'arrêt du conteneur. Utile pour les données sensibles temporaires.

```
1 docker run -d \
2     --name session-app \
3     --tmpfs /tmp:rw,size=64m \
4     mon-app
```

Listing 6.3 – Utilisation de tmpfs

6.3 Stratégies de sauvegarde

```
1 # Sauvegarder un volume
2 docker run --rm \
```

```
3     -v mes-donnees:/data \
4     -v $(pwd):/backup \
5     alpine \
6     tar czf /backup/backup.tar.gz -C /data .
7
8 # Restaurer un volume
9 docker run --rm \
10    -v mes-donnees:/data \
11    -v $(pwd):/backup \
12    alpine \
13    tar xzf /backup/backup.tar.gz -C /data
```

Listing 6.4 – Sauvegarde et restauration de volumes

Chapitre 7

Docker Compose

7.1 Introduction à Docker Compose

Docker Compose est un outil qui permet de définir et gérer des applications multi-conteneurs. Avec un seul fichier YAML (`docker-compose.yml`), vous définissez l'ensemble des services de votre application.

[i] Information

À partir de Docker Compose V2, la commande est `docker compose` (sans tiret), intégrée directement dans Docker CLI. L'ancienne commande `docker-compose` (avec tiret) correspond à la V1.

7.2 Structure d'un fichier `docker-compose.yml`

```
1 # Version du format (facultatif en V2+)
2 version: '3.8'
3
4 services:
5   # Définition des services (conteneurs)
6   nom-service:
7     image: nom-image:tag          # OU
8     build:
9       context: .
10      dockerfile: Dockerfile
11     container_name: mon-conteneur
12     restart: unless-stopped
13     ports:
14       - "hote:conteneur"
15     environment:
16       - VARIABLE=valeur
17     env_file:
18       - .env
19     volumes:
20       - volume-nomme:/chemin
```

```
21     - ./local:/conteneur
22   networks:
23     - mon-reseau
24   depends_on:
25     - autre-service
26   healthcheck:
27     test: ["CMD", "curl", "-f", "http://localhost"]
28     interval: 30s
29     timeout: 10s
30     retries: 3
31
32   volumes:
33     # Definition des volumes
34     volume-nomme:
35
36   networks:
37     # Definition des reseaux
38     mon-reseau:
39     driver: bridge
```

Listing 7.1 – Structure de base d'un docker-compose.yml

7.3 Exemple complet : Stack Web LAMP

```
1  version: '3.8'
2
3  services:
4    # Base de donnees MySQL
5    db:
6      image: mysql:8.0
7      container_name: wordpress-db
8      restart: always
9      environment:
10        MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
11        MYSQL_DATABASE: wordpress
12        MYSQL_USER: ${DB_USER}
13        MYSQL_PASSWORD: ${DB_PASSWORD}
14      volumes:
15        - db-data:/var/lib/mysql
16      networks:
17        - wp-network
18      healthcheck:
19        test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
20        interval: 10s
21        timeout: 5s
22        retries: 5
23
24    # Application WordPress
25    wordpress:
26      image: wordpress:6.4-php8.2-apache
```

```

27     container_name: wordpress-app
28     restart: always
29     ports:
30       - "80:80"
31     environment:
32       WORDPRESS_DB_HOST: db:3306
33       WORDPRESS_DB_USER: ${DB_USER}
34       WORDPRESS_DB_PASSWORD: ${DB_PASSWORD}
35       WORDPRESS_DB_NAME: wordpress
36     volumes:
37       - wp-content:/var/www/html/wp-content
38     networks:
39       - wp-network
40     depends_on:
41       db:
42         condition: service_healthy
43
44     # Interface d'administration
45     adminer:
46       image: adminer:latest
47       container_name: adminer
48       restart: always
49       ports:
50       - "8080:8080"
51       networks:
52       - wp-network
53
54     volumes:
55       db-data:
56       wp-content:
57
58     networks:
59     wp-network:
60       driver: bridge

```

Listing 7.2 – docker-compose.yml pour une stack WordPress

7.4 Commandes Docker Compose

```

1  # Demarrer tous les services (en avant-plan)
2  docker compose up
3
4  # Demarrer en arriere-plan
5  docker compose up -d
6
7  # Rebuilder et demarrer
8  docker compose up -d --build
9
10 # Arrêter les services
11 docker compose stop

```

```
12
13 # Arrêter et supprimer les conteneurs
14 docker compose down
15
16 # Arrêter, supprimer conteneurs ET volumes
17 docker compose down -v
18
19 # Lister les conteneurs du projet
20 docker compose ps
21
22 # Voir les logs
23 docker compose logs
24 docker compose logs -f wordpress      # logs d'un service
    spécifique
25
26 # Executer une commande dans un service
27 docker compose exec wordpress bash
28
29 # Scaler un service
30 docker compose up -d --scale wordpress=3
31
32 # Valider le fichier compose
33 docker compose config
```

Listing 7.3 – Commandes essentielles Docker Compose

Chapitre 8

Registres Docker et Distribution d'Images

8.1 Docker Hub

Docker Hub est le registre public par défaut de Docker. Il offre :

- Des dépôts publics gratuits (illimités)
- Des dépôts privés (limités selon le plan)
- Des webhooks et intégrations CI/CD
- Des builds automatiques depuis GitHub/GitLab

8.1.1 Publier une image sur Docker Hub

```
1 # 1. Se connecter a Docker Hub
2 docker login
3
4 # 2. Tagger l'image avec votre nom d'utilisateur
5 docker tag mon-app:v1.0 elkhlifi/mon-app:v1.0
6 docker tag mon-app:v1.0 elkhlifi/mon-app:latest
7
8 # 3. Pousser l'image
9 docker push elkhlifi/mon-app:v1.0
10 docker push elkhlifi/mon-app:latest
11
12 # 4. Se deconnecter
13 docker logout
```

Listing 8.1 – Publication sur Docker Hub

8.2 Autres registres

8.2.1 GitHub Container Registry (GHCR)

```
1 # Authentification
2 echo $GITHUB_TOKEN | docker login ghcr.io \
3   -u USERNAME --password-stdin
```

```
4
5 # Tagger pour GHCR
6 docker tag mon-app:v1.0 ghcr.io/elkhlifi/mon-app:v1.0
7
8 # Pousser
9 docker push ghcr.io/elkhlifi/mon-app:v1.0
```

Listing 8.2 – Utilisation de GHCR

8.2.2 Registre privé auto-hébergé

```
1 # Lancer le registre officiel
2 docker run -d \
3     --name registry \
4     --restart always \
5     -p 5000:5000 \
6     -v registry-data:/var/lib/registry \
7     registry:2
8
9 # Utiliser le registre local
10 docker tag mon-app:v1.0 localhost:5000/mon-app:v1.0
11 docker push localhost:5000/mon-app:v1.0
12 docker pull localhost:5000/mon-app:v1.0
```

Listing 8.3 – Déployer un registre privé avec Docker

Chapitre 9

Sécurité Docker

9.1 Principes de sécurité fondamentaux

La sécurité est un aspect critique de tout déploiement Docker. Cette section couvre les meilleures pratiques pour sécuriser vos conteneurs et votre hôte Docker.

[!] Avertissement

Par défaut, le daemon Docker s'exécute avec les privilèges root. Une mauvaise configuration peut exposer votre système à des risques de sécurité importants.

9.2 Sécurité des images

9.2.1 Utiliser des images de confiance

- Préférer les images **officielles** ou **vérifiées**
- Utiliser des **tags spécifiques** (éviter `:latest`)
- Scanner les images avec `docker scout` ou Trivy
- Vérifier les signatures avec Docker Content Trust (DCT)

```
1 # Avec Docker Scout (integre)
2 docker scout cves nginx:latest
3
4 # Avec Trivy (outil externe populaire)
5 docker run --rm \
6     -v /var/run/docker.sock:/var/run/docker.sock \
7     aquasec/trivy image nginx:alpine
8
9 # Activer Docker Content Trust
10 export DOCKER_CONTENT_TRUST=1
11 docker pull nginx      # verifie la signature
```

Listing 9.1 – Scanner une image pour les vulnérabilités

9.3 Sécurité des conteneurs

9.3.1 Principe du moindre privilège

```
1 # Eviter d'exécuter en root
2 FROM alpine:3.18
3
4 RUN addgroup -S appgroup && \
5     adduser -S appuser -G appgroup
6
7 # ... installer l'application ...
8
9 USER appuser      # Passer a l'utilisateur non-root
10
11 # Conteneur en lecture seule
12 docker run --read-only \
13     --tmpfs /tmp \
14     mon-app
15
16 # Supprimer toutes les capabilities
17 docker run \
18     --cap-drop ALL \
19     --cap-add NET_BIND_SERVICE \
20     mon-app
21
22 # Mode securise
23 docker run \
24     --security-opt no-new-privileges:true \
25     --security-opt seccomp=/etc/docker/seccomp.json \
26     mon-app
```

Listing 9.2 – Sécurité des conteneurs en pratique

9.3.2 Limiter les ressources

```
1 docker run -d \
2     --name app-prod \
3     --memory="256m" \
4     --memory-swap="512m" \
5     --cpus="0.5" \
6     --pids-limit=100 \
7     --ulimit nofile=1024:1024 \
8     mon-app
```

Listing 9.3 – Limitation des ressources

9.4 Sécurité du daemon Docker

```
1 # /etc/docker/daemon.json
2 {
```

```
3  "icc": false,  
4  "usersns-remap": "default",  
5  "no-new-privileges": true,  
6  "log-driver": "json-file",  
7  "log-opts": {  
8      "max-size": "10m",  
9      "max-file": "3"  
10 },  
11 "live-restore": true  
12 }
```

Listing 9.4 – Configuration sécurisée du daemon

Chapitre 10

Docker en Production

10.1 Bonnes pratiques de production

Déployer Docker en production nécessite une attention particulière à la fiabilité, aux performances et à la maintenabilité.

10.2 Logging et monitoring

10.2.1 Configuration des logs

```
1 # Configurer le driver de logs
2 docker run -d \
3     --log-driver json-file \
4     --log-opt max-size=10m \
5     --log-opt max-file=3 \
6     --name prod-app \
7     mon-app
8
9 # Envoyer les logs vers syslog
10 docker run -d \
11     --log-driver syslog \
12     --log-opt syslog-address=tcp://log-server:514 \
13     mon-app
```

Listing 10.1 – Gestion avancée des logs

10.2.2 Stack de monitoring : Prometheus + Grafana

```
1 version: '3.8'
2
3 services:
4   prometheus:
5     image: prom/prometheus:latest
6     container_name: prometheus
7     ports:
```

```

8     - "9090:9090"
9     volumes:
10    - ./prometheus.yml:/etc/prometheus/prometheus.yml
11    - prometheus-data:/prometheus
12    command:
13    - '--config.file=/etc/prometheus/prometheus.yml'
14
15    grafana:
16      image: grafana/grafana:latest
17      container_name: grafana
18      ports:
19      - "3000:3000"
20      environment:
21      - GF_SECURITY_ADMIN_PASSWORD=admin123
22      volumes:
23      - grafana-data:/var/lib/grafana
24      depends_on:
25      - prometheus
26
27    cadvisor:
28      image: gcr.io/cadvisor/cadvisor:latest
29      container_name: cadvisor
30      ports:
31      - "8080:8080"
32      volumes:
33      - /:/rootfs:ro
34      - /var/run:/var/run:ro
35      - /sys:/sys:ro
36      - /var/lib/docker/containers:/var/lib/docker:ro
37
38    volumes:
39      prometheus-data:
40      grafana-data:

```

Listing 10.2 – docker-compose.yml pour le monitoring

10.3 Haute disponibilité avec Docker Swarm

Docker Swarm est la solution native d'orchestration de Docker pour la haute disponibilité et la mise à l'échelle.

```

1  # Initialiser le Swarm (sur le noeud manager)
2  docker swarm init --advertise-addr 192.168.1.10
3
4  # Ajouter des noeuds workers (sur chaque worker)
5  docker swarm join \
6    --token <TOKEN> \
7    192.168.1.10:2377
8
9  # Deployer un service
10 docker service create \

```

```
11  --name web-service \  
12  --replicas 3 \  
13  -p 80:80 \  
14  nginx:alpine  
15  
16  # Mettre a l'echelle  
17  docker service scale web-service=5  
18  
19  # Mise a jour progressive (rolling update)  
20  docker service update \  
21  --image nginx:1.25 \  
22  --update-parallelism 1 \  
23  --update-delay 10s \  
24  web-service  
25  
26  # Lister les services  
27  docker service ls  
28  
29  # Voir les replicas  
30  docker service ps web-service
```

Listing 10.3 – Configuration d'un cluster Docker Swarm

10.4 CI/CD avec Docker

10.4.1 Intégration dans GitHub Actions

```
1  # .github/workflows/docker-build.yml  
2  name: Build and Push Docker Image  
3  
4  on:  
5    push:  
6      branches: [ main ]  
7    pull_request:  
8      branches: [ main ]  
9  
10 jobs:  
11   build:  
12     runs-on: ubuntu-latest  
13     steps:  
14       - name: Checkout code  
15         uses: actions/checkout@v4  
16  
17       - name: Set up Docker Buildx  
18         uses: docker/setup-buildx-action@v3  
19  
20       - name: Login to Docker Hub  
21         uses: docker/login-action@v3  
22         with:  
23           username: ${ secrets.DOCKER_USERNAME }}
```



```
24     password: ${ secrets.DOCKER_TOKEN }}
25
26 - name: Build and push
27   uses: docker/build-push-action@v5
28   with:
29     context: .
30     push: true
31     tags: |
32       elkhlifi/mon-app:latest
33       elkhlifi/mon-app:${ github.sha }}
34   cache-from: type=gha
35   cache-to: type=gha,mode=max
```

Listing 10.4 – Workflow GitHub Actions pour Docker

Chapitre 11

Cas Pratiques et Exercices

11.1 TP1 : Déployer un site web statique

Objectif : Créer une image Docker servant un site web statique avec Nginx.

```
1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4     <meta charset="UTF-8">
5     <title>Mon Site Docker</title>
6     <style>
7         body { font-family: Arial, sans-serif;
8                 text-align: center;
9                 background: #0d73d1; color: white; }
10        h1 { margin-top: 100px; font-size: 3em; }
11        p { font-size: 1.2em; }
12    </style>
13 </head>
14 <body>
15     <h1>Bienvenue sur Docker!</h1>
16     <p>Déployé par EL KHLIFI Imad</p>
17 </body>
18 </html>
```

Listing 11.1 – index.html

```
1 FROM nginx:alpine
2
3 LABEL maintainer="EL KHLIFI Imad"
4
5 COPY index.html /usr/share/nginx/html/index.html
6
7 EXPOSE 80
8
9 HEALTHCHECK --interval=30s --timeout=3s \
10     CMD wget -q -O- http://localhost/ || exit 1
```

Listing 11.2 – Dockerfile pour le site web

```

1 # Construire l'image
2 docker build -t mon-site:v1.0 .
3
4 # Lancer le conteneur
5 docker run -d -p 8080:80 --name mon-site mon-site:v1.0
6
7 # Verifier
8 curl http://localhost:8080
9
10 # Ouvrir dans le navigateur : http://localhost:8080

```

Listing 11.3 – Commandes de déploiement

11.2 TP2 : Stack complète avec Docker Compose

Objectif : Déployer une application Python Flask avec Redis pour le comptage de vues.

```

1 from flask import Flask
2 import redis
3 import os
4
5 app = Flask(__name__)
6 r = redis.Redis(host='redis', port=6379)
7
8 @app.route('/')
9 def index():
10     count = r.incr('hits')
11     return f'''
12     <h1>Bienvenue!</h1>
13     <p>Cette page a ete visitee {count} fois.</p>
14     <p>Développé par <strong>EL KHLIFI Imad</strong></p>
15     '''
16
17 if __name__ == '__main__':
18     app.run(host='0.0.0.0', port=5000, debug=False)

```

Listing 11.4 – app.py Application Flask

```

1 FROM python:3.11-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6 RUN pip install --no-cache-dir -r requirements.txt
7
8 COPY app.py .
9
10 RUN useradd -m -r appuser && chown appuser /app
11 USER appuser

```

```
12
13 EXPOSE 5000
14
15 CMD ["python", "app.py"]
```

Listing 11.5 – Dockerfile pour Flask

```
1 version: '3.8'
2
3 services:
4   web:
5     build: .
6     container_name: flask-app
7     ports:
8       - "5000:5000"
9     networks:
10      - app-network
11     depends_on:
12      - redis
13     restart: unless-stopped
14
15   redis:
16     image: redis:7-alpine
17     container_name: app-redis
18     networks:
19      - app-network
20     volumes:
21      - redis-data:/data
22     restart: unless-stopped
23
24 networks:
25   app-network:
26
27 volumes:
28   redis-data:
```

Listing 11.6 – docker-compose.yml

```
1 # Demarrer la stack
2 docker compose up -d --build
3
4 # Verifier les services
5 docker compose ps
6
7 # Tester l'application
8 curl http://localhost:5000
9
10 # Voir les logs
11 docker compose logs -f web
12
13 # Arrêter proprement
14 docker compose down
```

Listing 11.7 – Déploiement et test

11.3 TP3 : Exportation et portabilité

Objectif : Démontrer la portabilité en exportant et important une image.

```
1 # MACHINE SOURCE : Sauvegarder l'image
2 docker save -o mon-app.tar mon-app:v1.0
3
4 # Verifier la taille du fichier
5 ls -lh mon-app.tar
6
7 # Transfert via SCP vers une autre machine
8 scp mon-app.tar user@serveur:/home/user/
9
10 # MACHINE DESTINATION : Charger l'image
11 docker load -i mon-app.tar
12
13 # Verifier que l'image est disponible
14 docker images | grep mon-app
15
16 # Lancer le conteneur
17 docker run -d -p 8080:80 mon-app:v1.0
18
19 # Tester
20 curl http://localhost:8080
```

Listing 11.8 – Export

Chapitre 12

Référence des Commandes et Glossaire

12.1 Cheat Sheet Docker

Gestion des images	
<code>docker pull [image]</code>	Télécharger une image
<code>docker push [image]</code>	Pousser vers un registre
<code>docker build -t [tag] .</code>	Construire une image
<code>docker images</code>	Lister les images locales
<code>docker rmi [image]</code>	Supprimer une image
<code>docker image prune</code>	Supprimer les images inutilisées
Gestion des conteneurs	
<code>docker run -d [image]</code>	Lancer en arrière-plan
<code>docker run -it [image] sh</code>	Terminal interactif
<code>docker ps</code>	Lister conteneurs actifs
<code>docker ps -a</code>	Tous les conteneurs
<code>docker stop [id]</code>	Arrêter un conteneur
<code>docker rm [id]</code>	Supprimer un conteneur
<code>docker exec -it [id] sh</code>	Entrer dans un conteneur
<code>docker logs -f [id]</code>	Suivre les logs
<code>docker stats</code>	Statistiques en temps réel
<code>docker inspect [id]</code>	Configuration complète
Volumes et Réseaux	
<code>docker volume create [nom]</code>	Créer un volume
<code>docker volume ls</code>	Lister les volumes
<code>docker network create [nom]</code>	Créer un réseau
<code>docker network ls</code>	Lister les réseaux
Docker Compose	
<code>docker compose up -d</code>	Démarrer la stack
<code>docker compose down</code>	Arrêter la stack
<code>docker compose ps</code>	État des services
<code>docker compose logs -f</code>	Suivre les logs
<code>docker compose exec [svc] sh</code>	Entrer dans un service
Nettoyage	
<code>docker system prune</code>	Nettoyer les ressources inutilisées
<code>docker system prune -a</code>	Nettoyage complet
<code>docker system df</code>	Espace utilisé par Docker

12.2 Glossaire Docker

Image	Modèle en lecture seule contenant un système de fichiers et la configuration pour créer des conteneurs. Composée de couches superposées.
Conteneur	Instance en cours d'exécution d'une image. Environnement isolé et léger partageant le noyau de l'hôte.

Dockerfile	Fichier texte contenant les instructions pour construire une image Docker de manière reproductible.
Registry	Service de stockage et distribution d'images Docker. Docker Hub est le registre public par défaut.
Docker Compose	Outil pour définir et gérer des applications multi-conteneurs via un fichier YAML.
Volume	Mécanisme de persistance des données géré par Docker, indépendant du cycle de vie des conteneurs.
Bind Mount	Montage d'un répertoire de l'hôte dans un conteneur. Utile en développement.
Layer (couche)	Unité immuable dans une image Docker. Chaque instruction Dockerfile crée une nouvelle couche.
Tag	Étiquette permettant d'identifier une version spécifique d'une image (<code>nginx:1.25</code>).
Docker Hub	Registre public officiel de Docker avec des millions d'images disponibles.
Docker Engine	Application client-serveur comprenant le daemon Docker, l'API REST et le CLI.
Namespace	Mécanisme Linux d'isolation des ressources utilisé par Docker pour isoler les conteneurs.
cgroup	Control Group Linux permettant de limiter et contrôler les ressources allouées aux conteneurs.
Swarm	Mode natif d'orchestration de Docker permettant de gérer un cluster de noeuds Docker.
Docker Scout	Outil intégré à Docker pour analyser les vulnérabilités de sécurité dans les images.
ENTRYPOINT	Instruction Dockerfile définissant le point d'entrée principal du conteneur, non remplaçable par défaut.
CMD	Instruction Dockerfile définissant la commande par défaut, remplaçable lors du <code>docker run</code> .
Multi-stage build	Technique Dockerfile permettant d'utiliser plusieurs étapes de construction pour créer des images légères.

12.3 Ressources et liens utiles

- Documentation officielle : <https://docs.docker.com>
- Docker Hub : <https://hub.docker.com>
- Play with Docker : <https://labs.play-with-docker.com>
- Docker Blog : <https://www.docker.com/blog>
- GitHub Docker : <https://github.com/docker>

[DOCKER]

Docker : Guide Complet

Ce guide vous a accompagné à travers tous les aspects fondamentaux et avancés de Docker.

*De la conteneurisation de base à la production,
en passant par Docker Compose, les réseaux,
les volumes et la sécurité, vous disposez maintenant
des connaissances nécessaires pour déployer
des applications robustes avec Docker.*

EL KHLIFI Imad

PhDc | Data Scientist | Data Analyst | AI